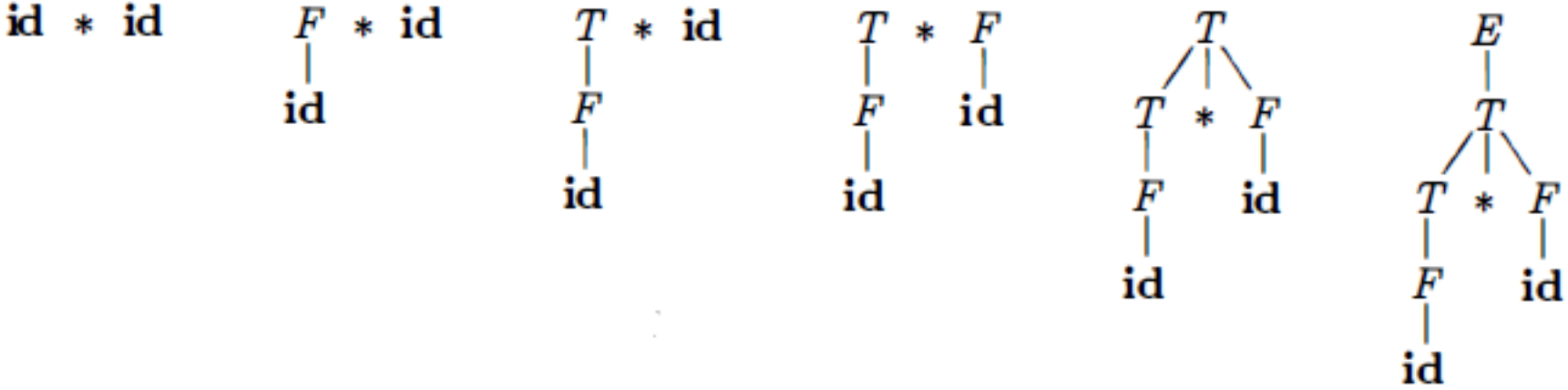
Bottom-Up Parsing

Acknowledgement

- Alfred V Aho, Monica S. Lam, Ravi Sethi, Jeffrey D Ullman- “Compilers- Principles, Techniques and Tools”

Bottom-Up Parsing

- Construction of a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top)



Bottom-Up Parsing

- Bottom-up parsing as the process of "reducing" a string “w” to the start symbol of the grammar.
- At each reduction step, a specific substring matching the body of a production is replaced by the nonterminal at the head of that production.
- The key decisions during bottom-up parsing are about when to reduce and about what production to apply

Bottom-Up Parsing

- Bottom-up parsing during a left-to-right scan of the input constructs a rightmost derivation in reverse.
- A "handle" is a substring that matches the body of a production, and whose reduction represents one step along the reverse of a rightmost derivation.

$$
\begin{array}{lcl}
E & \rightarrow & E + T \mid T \\
T & \rightarrow & T * F \mid F \\
F & \rightarrow & (E) \mid \mathbf{id}
\end{array}
$$

RIGHT SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \rightarrow \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \rightarrow F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \rightarrow \mathbf{id}$
$T * F$	$T * F$	$E \rightarrow T * F$

Shift-Reduce Parsing

- Shift-reduce parsing is a form of bottom-up parsing in which a stack holds grammar symbols and an input buffer holds the rest of the string to be parsed.

Shift-Reduce Parsing

- There are four possible actions of a shift-parser action:

1. **Shift** : The next input symbol is shifted onto the top of the stack.
2. **Reduce**: Replace the handle on the top of the stack by the non-terminal.
3. **Accept**: Successful completion of parsing.
4. **Error**: Parser discovers a syntax error, and calls an error recovery routine.

- Initial stack just contains only the end-marker $.
- The end of the input string is marked by the end-marker $.

Shift-Reduce Parsing

- The handle will always eventually appear on top of the stack, never inside.

$$
\begin{array}{lcl}
E & \rightarrow & E + T \mid T \\
T & \rightarrow & T * F \mid F \\
F & \rightarrow & (E) \mid \mathbf{id}
\end{array}
$$

STACK	INPUT	ACTION
$	$\mathbf{id}_1 * \mathbf{id}_2$ $	shift
$ $\mathbf{id}_1$	$* \mathbf{id}_2$ $	reduce by $F \rightarrow \mathbf{id}$
$ F	$* \mathbf{id}_2$ $	reduce by $T \rightarrow F$
$ T	$* \mathbf{id}_2$ $	shift
$ $T *$	$\mathbf{id}_2$ $	shift
$ $T * \mathbf{id}_2$	$	reduce by $F \rightarrow \mathbf{id}$
$ $T * F$	$	reduce by $T \rightarrow T * F$
$ T	$	reduce by $E \rightarrow T$
$ E	$	accept

Shift-Reduce Parsing

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

Right-Most Derivation of `id+id*id`

$E \Rightarrow E+T \Rightarrow E+T*F \Rightarrow E+T*id \Rightarrow E+F*id$

$\Rightarrow E+id*id \Rightarrow T+id*id \Rightarrow F+id*id \Rightarrow id+id*id$

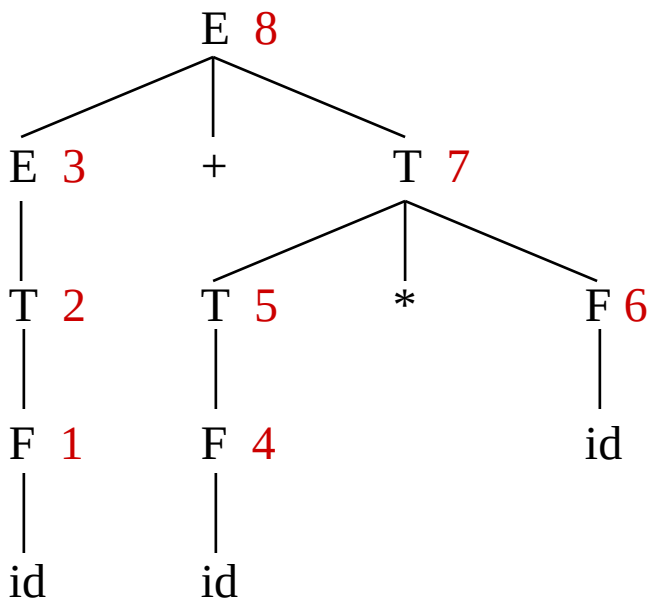
Right-Most Sentential Form	Reducing Production
id+id*id	$F \rightarrow id$
F+id*id	$T \rightarrow F$
T+id*id	$E \rightarrow T$
E+id*id	$F \rightarrow id$
E+F*id	$T \rightarrow F$
E+T*id	$F \rightarrow id$
E+T*F	$T \rightarrow T*F$
E+T	$E \rightarrow E+T$
E	

Handles are red and underlined in the right-sentential forms.

Shift-Reduce Parsing

Stack	**Input**	**Action**
$	id+id*id$	shift
$id	+id*id$	reduce by F → id
$F	+id*id$	reduce by T → F
$T	+id*id$	reduce by E → T
$E	+id*id$	shift
$E+	id*id$	shift
$E+id	*id$	reduce by F → id
$E+F	*id$	reduce by T → F
$E+T	*id$	shift
$E+T*	id$	shift
$E+T*id	$	reduce by F → id
$E+T*F	$	reduce by T → T*F
$E+T	$	reduce by E → E+T
$E	$	accept

Parse Tree



Conflicts During Shift-Reduce Parsing

- There are context-free grammars for which shift-reduce parsers cannot be used.
- Stack contents and the next input symbol may not decide action:
  - **shift/reduce conflict**: Whether make a shift operation or a reduction.
  - **reduce/reduce conflict**: The parser cannot decide which of several reductions to make.
- If a shift-reduce parser cannot be used for a grammar, that grammar is called as non-LR(k) grammar.

left to right scanning

right-most derivation

k lookahead

- An ambiguous grammar can never be a LR grammar.

Conflicts During Shift-Reduce Parsing

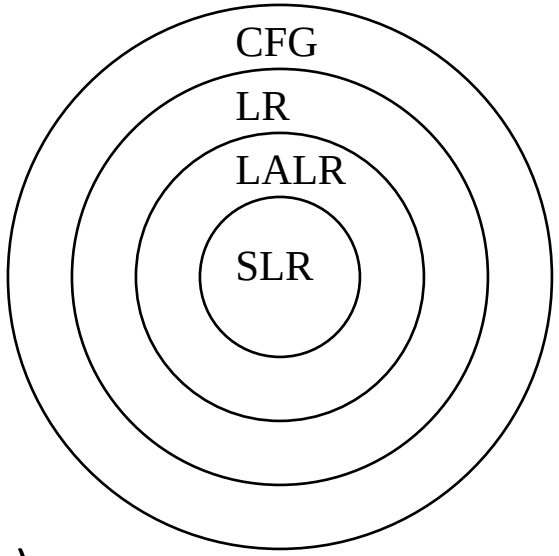
$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{if}\ expr\ \textbf{then}\ stmt \\
 & | & \textbf{if}\ expr\ \textbf{then}\ stmt\ \textbf{else}\ stmt \\
 & | & \textbf{other}
\end{array}
$$

STACK	INPUT
$\cdots$ **if** *expr* **then** *stmt*	**else** $\cdots$ \$

Shift-Reduce Parsing

- There are two main categories of shift-reduce parsers

1. **Operator-Precedence Parser**
   - simple, but only a small class of grammars.



2. **LR-Parsers**
   - covers wide range of grammars.
     - SLR – simple LR parser
     - LR – most general LR parser
     - LALR – intermediate LR parser (lookhead LR parser)
   - SLR, LR and LALR work same, only their parsing tables are different.

Introduction to LR Parsing: Simple LR

- The most powerful shift-reduce parsing (yet efficient) is:

LR(k) parsing

left to right scanning

right-most derivation

k lookahead (k is omitted ➔ it is 1)

- LR parsing is attractive because:
  - LR parsing is most general non-backtracking shift-reduce parsing, yet it is still efficient.
  - The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive parsers.

    LL(1)-Grammars $\subset$ LR(1)-Grammars
  - An LR-parser can detect a syntactic error as soon as it is possible to do so a left-to-right scan of the input.

Items and the LR(O) Automaton

- An LR parser makes shift-reduce decisions by maintaining states to keep track of where we are in a parse.
- **States represent sets of "items."**
- An LR(O) item (item for short) of a grammar G is a production of G with a dot at some position of the body.

$$A \rightarrow \cdot XYZ$$
$$A \rightarrow X \cdot YZ$$
$$A \rightarrow XY \cdot Z$$
$$A \rightarrow XYZ \cdot$$

- The production A → ε generates only one item, A → .
- An item indicates how much of a production we have seen at a given point in the parsing process

Items and the LR(O) Automaton

- One collection of sets of LR(O) items, called the canonical LR(O) collection, provides the basis for constructing a deterministic finite automaton that is used to make parsing decisions.
- Such an automaton is called an LR (O) automaton.
- In particular, each state of the LR(O) automaton represents a set of items in the canonical LR(O) collection.
- To construct the canonical LR(O) collection for a grammar, define an **augmented grammar** and two functions, **CLOSURE** and **GOTO** .
- *Augmented Grammar*:

  G’ is G with a new production rule $S' \rightarrow S$ where S’ is the new starting symbol.

Closure of Item Sets

- If ***I*** is a set of LR(0) items for a grammar G, then ***closure(I)*** is the set of LR(0) items constructed from I by the two rules:

  1. add every item in I to CLOSURE(I).
  2. If $A \rightarrow \alpha.B\beta$ is in CLOSURE(I) and $B \rightarrow \Upsilon$ is a production, then add the item $B \rightarrow .\Upsilon$ CLOSURE(I)

  Apply this rule until no more new LR(0) items can be added to closure(I).

Closure of Item Sets

```
SetOfItems CLOSURE(I) {
    J = I;
    repeat
        for ( each item A → α·Bβ in J )
            for ( each production B → γ of G )
                if ( B → ·γ is not in J )
                    add B → ·γ to J;
    until no more items are added to J on one round;
    return J;
}
```

Closure of Item Sets

$E' \rightarrow E$

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow T*F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

closure({$E' \rightarrow \bullet E$}) =

{ $E' \rightarrow \bullet E$ ← kernel items

$E \rightarrow \bullet E+T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T*F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet id$ }

Closure of Item Sets

- We divide all the sets of items of interest into two classes:

1. Kernel items : the initial item, $S' \rightarrow \cdot S$, and all items whose dots are not at the left end.
2. Nonkernel items : all items with their dots at the left end, except for $S' \rightarrow \cdot S$.

The Function GOTO

- GOTO (I, X) where I is a set of items and X is a grammar symbol.
- GOTO (I, X) is defined to be the closure of the set of all items [A $\rightarrow$ αX·β] such that [A $\rightarrow$ α.Xβ] is in I.
- If I is the set of two items { [E' $\rightarrow$ E·] , [E $\rightarrow$ E· + T] } , then GOTO(I, +) contains the items

$$
\begin{aligned}
E &\rightarrow E + \cdot T \\
T &\rightarrow \cdot T * F \\
T &\rightarrow \cdot F \\
F &\rightarrow \cdot (E) \\
F &\rightarrow \cdot \mathbf{id}
\end{aligned}
$$

The Function GOTO

$I = \{ E' \rightarrow \bullet E,\ E \rightarrow \bullet E+T,\ E \rightarrow \bullet T,\ T \rightarrow \bullet T*F,\ T \rightarrow \bullet F,$
$F \rightarrow \bullet (E),\ F \rightarrow \bullet id \}$

goto(I,E) = $\{ E' \rightarrow E \bullet,\ E \rightarrow E \bullet +T \}$

goto(I,T) = $\{ E \rightarrow T \bullet,\ T \rightarrow T \bullet *F \}$

goto(I,F) = $\{ T \rightarrow F \bullet \}$

goto(I,() = $\{ F \rightarrow (\bullet E),\ E \rightarrow \bullet E+T,\ E \rightarrow \bullet T,\ T \rightarrow \bullet T*F,\ T \rightarrow \bullet F,$
$F \rightarrow \bullet (E),\ F \rightarrow \bullet id \}$

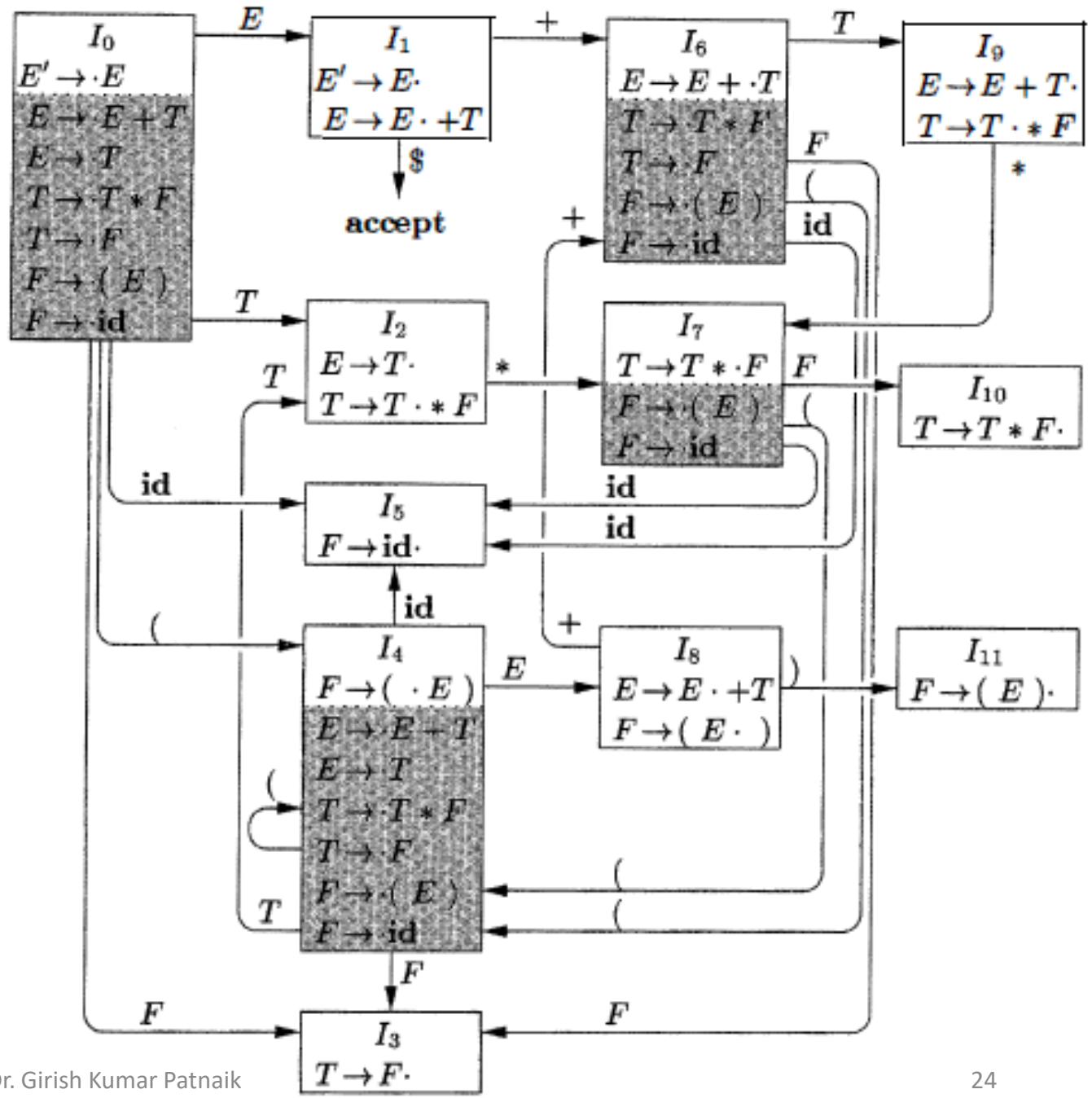
goto(I,id) = $\{ F \rightarrow id \bullet \}$

Construction of Canonical Collection of sets of LR(O) items for an augmented grammar G'

```
void items(G') {
    C = CLOSURE({[S' → ·S]});
    repeat
        for ( each set of items I in C )
            for ( each grammar symbol X )
                if ( GOTO(I, X) is not empty and not in C )
                    add GOTO(I, X) to C;
    until no new sets of items are added to C on a round;
}
```

LR(O) automaton

$$
\begin{array}{lcl}
E & \rightarrow & E + T \mid T \\
T & \rightarrow & T * F \mid F \\
F & \rightarrow & (E) \mid \textbf{id}
\end{array}
$$



Use of the LR(O) Automaton

- Shift-reduce decisions can be made as follows.
- Suppose that the string Υ of grammar symbols takes the LR(O) automaton from the start state 0 to some state j .
  - Then, shift on next input symbol a if state j has a transition on “a”.
  - Otherwise, we choose to reduce; the items in state j will tell us which production to use.

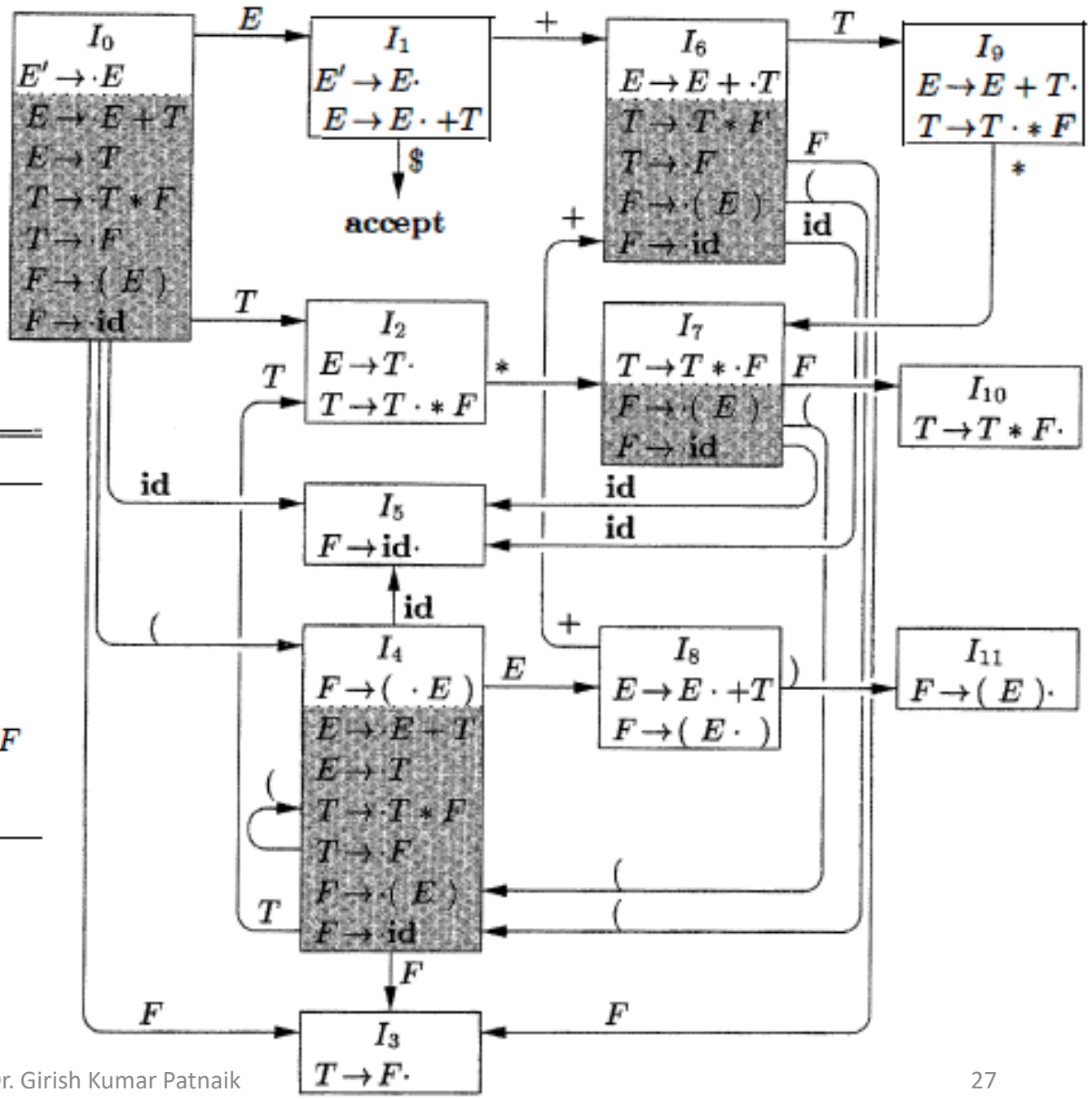
Use of the LR(O) Automaton

- Uses its stack to keep track of states as well as grammar symbols;

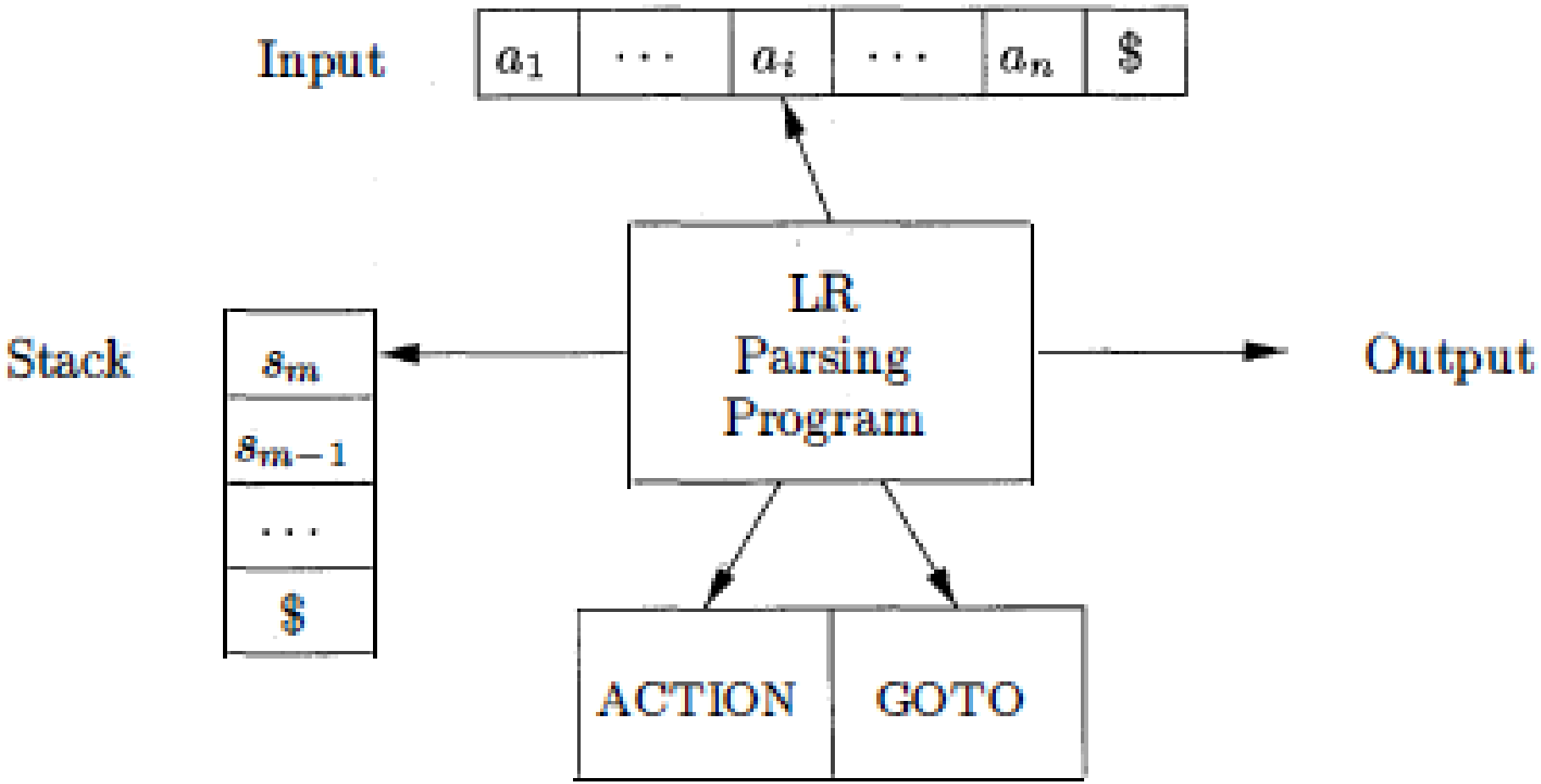
LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \rightarrow$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \rightarrow F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ $T *$	**id** \$	shift to 5
(6)	0 2 7 5	\$ $T *$ **id**	\$	reduce by $F \rightarrow$ **id**
(7)	0 2 7 10	\$ $T * F$	\$	reduce by $T \rightarrow T * F$
(8)	0 2	\$ T	\$	reduce by $E \rightarrow T$
(9)	0 1	\$ E	\$	accept

$$
\begin{array}{lcl}
E & \rightarrow & E + T \mid T \\
T & \rightarrow & T * F \mid F \\
F & \rightarrow & (E) \mid \textbf{id}
\end{array}
$$

LINE	STACK	SYMBOLS	INPUT	ACTION
(1)	0	\$	**id** $*$ **id** \$	shift to 5
(2)	0 5	\$ **id**	$*$ **id** \$	reduce by $F \rightarrow$ **id**
(3)	0 3	\$ F	$*$ **id** \$	reduce by $T \rightarrow F$
(4)	0 2	\$ T	$*$ **id** \$	shift to 7
(5)	0 2 7	\$ $T *$	**id** \$	shift to 5
(6)	0 2 7 5	\$ $T *$ **id**	\$	reduce by $F \rightarrow$ **id**
(7)	0 2 7 10	\$ $T * F$	\$	reduce by $T \rightarrow T * F$
(8)	0 2	\$ T	\$	reduce by $E \rightarrow T$
(9)	0 1	\$ E	\$	accept



The LR-Parsing Algorithm



Structure of the LR Parsing Table

- The parsing table consists of two parts: a parsing-action function ACTION and a goto function GOTO
- The ACTION function of ACTION[i, a] takes as arguments a state i and a terminal "a" (or $, the input endmarker) can have one of four forms:
  - Shift j , where j is a state. The action taken by the parser effectively shifts input "a" to the stack, but uses state j to represent "a" .
  - Reduce $A \rightarrow \beta$. The action of the parser effectively reduces β on the top of the stack to head A.
  - Accept. The parser accepts the input and finishes parsing;
  - Error. The parser discovers an error in its input and takes some corrective action.
- We extend the GOTO function, defined on sets of items, to states: if GOTo $[I_i , A] = I_j$, then GOTO also maps a state i and a nonterminal A to state j .

Behavior of the LR Parser

- A configuration of an LR parser is a pair:

$$(s_0 s_1 \cdots s_m,\ a_i a_{i+1} \cdots a_n \$)$$

- The next move of the parser is determined by reading a_i , the current input symbol, and S_m , the state on top of the stack, and then consulting the entry ACTION[S_m , a_i] in the parsing action table.
- If ACTION[S_m , a_i] = shift s, the parser executes a shift move; it shifts the next state S onto the stack, entering the configuration

$$(s_0 s_1 \cdots s_m s,\ a_{i+1} \cdots a_n \$)$$

Behavior of the LR Parser

- If ACTION[S_m , a_i] = reduce A $\rightarrow$ β, then the parser executes a reduce move, entering the configuration

$$(s_0 s_1 \cdots s_{m-r} s,\ \ a_i a_{i+1} \cdots a_n \$)$$

where r is the length of β, and s = GOTO[S_{m-r} , A] .

- Here the parser first popped r state symbols off the stack, exposing state S_{m-r} . The parser then pushed s, the entry for GOTO[S_{m-r}, A] , onto the stack. The current input symbol is not changed in a reduce move.

Behavior of the LR Parser

- If ACTION[S_m , a_i] = accept, parsing is completed
- If ACTION[S_m , a_i] = error, the parser has discovered an error and calls an error recovery routine

LR-parsing algorithm

```
let a be the first symbol of w$;
while(1) { /* repeat forever */
        let s be the state on top of the stack;
        if ( ACTION[s, a] = shift t ) {
                push t onto the stack;
                let a be the next input symbol;
        } else if ( ACTION[s, a] = reduce A → β ) {
                pop |β| symbols off the stack;
                let state t now be on top of the stack;
                push GOTO[t, A] onto the stack;
                output the production A → β;
        } else if ( ACTION[s, a] = accept ) break; /* parsing is done */
        else call error-recovery routine;
}
```

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

(1) $E \to E + T$
(2) $E \to T$
(3) $T \to T * F$
(4) $T \to F$
(5) $F \to (E)$
(6) $F \to \mathbf{id}$

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		**id** * **id** + **id** $	shift
(2)	0 5	**id**	* **id** + **id** $	reduce by $F \to \mathbf{id}$
(3)	0 3	F	* **id** + **id** $	reduce by $T \to F$
(4)	0 2	T	* **id** + **id** $	shift
(5)	0 2 7	T *	**id** + **id** $	shift
(6)	0 2 7 5	T * **id**	+ **id** $	reduce by $F \to \mathbf{id}$
(7)	0 2 7 10	$T * F$	+ **id** $	reduce by $T \to T * F$
(8)	0 2	T	+ **id** $	reduce by $E \to T$
(9)	0 1	E	+ **id** $	shift
(10)	0 1 6	E +	**id** $	shift
(11)	0 1 6 5	E + **id**	$	reduce by $F \to \mathbf{id}$
(12)	0 1 6 3	$E + F$	$	reduce by $T \to F$
(13)	0 1 6 9	$E + T$	$	reduce by $E \to E + T$
(14)	0 1	E	$	accept

Constructing SLR-Parsing Tables

1. Construct $C = \{I_0, I_1, \ldots, I_n\}$, the collection of sets of LR(0) items for G'.

2. State i is constructed from I_i. The parsing actions for state i are determined as follows:

   (a) If $[A \to \alpha \cdot a\beta]$ is in I_i and $\text{GOTO}(I_i, a) = I_j$, then set $\text{ACTION}[i, a]$ to "shift j." Here a must be a terminal.

   (b) If $[A \to \alpha \cdot]$ is in I_i, then set $\text{ACTION}[i, a]$ to "reduce $A \to \alpha$" for all a in $\text{FOLLOW}(A)$; here A may not be S'.

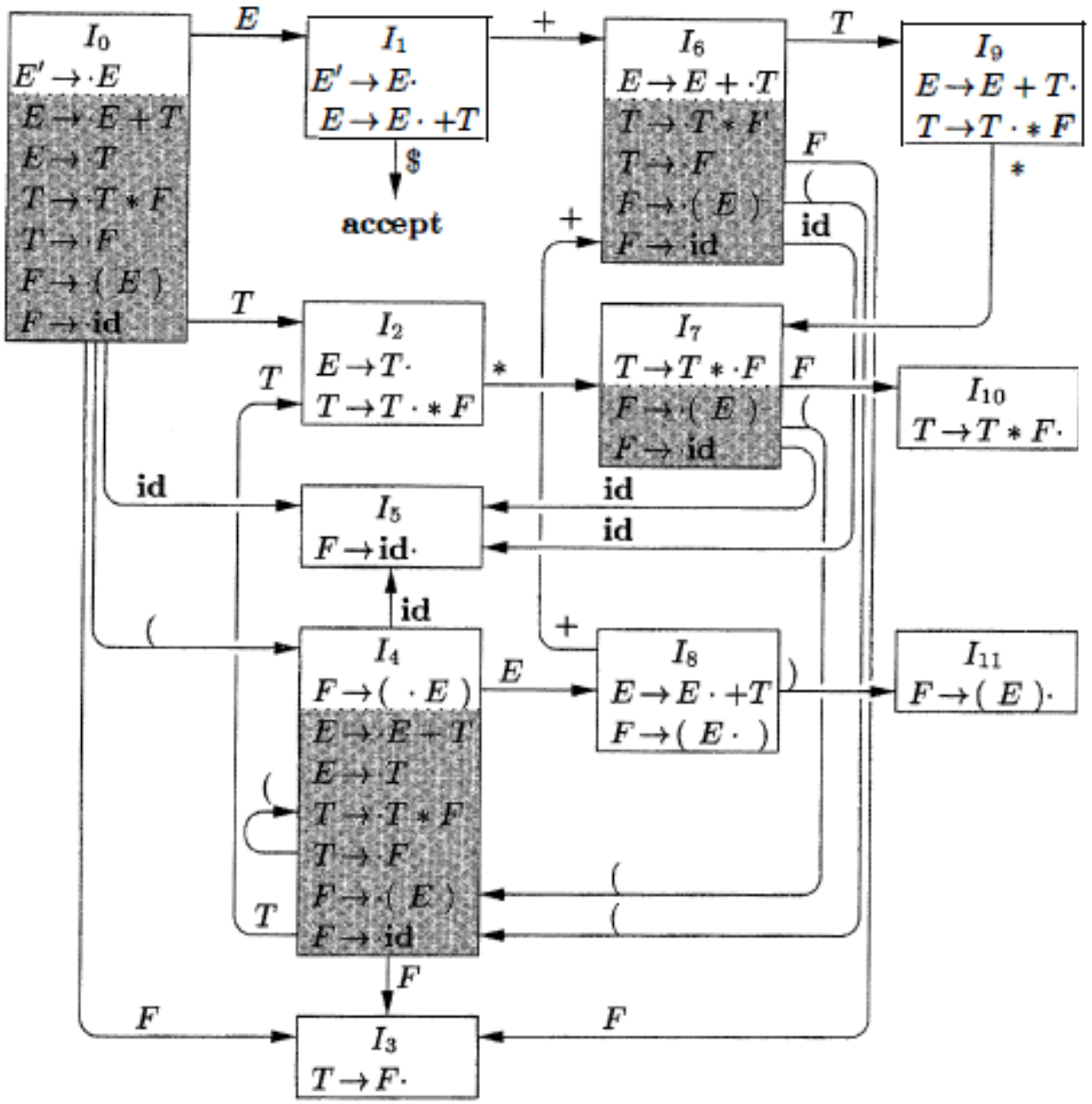
   (c) If $[S' \to S \cdot]$ is in I_i, then set $\text{ACTION}[i, \$]$ to "accept."

   If any conflicting actions result from the above rules, we say the grammar is not SLR(1). The algorithm fails to produce a parser in this case.

3. The goto transitions for state i are constructed for all nonterminals A using the rule: If $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}[i, A] = j$.

4. All entries not defined by rules (2) and (3) are made "error."

5. The initial state of the parser is the one constructed from the set of items containing $[S' \to \cdot S]$.



(1) $E \to E + T$
(2) $E \to T$
(3) $T \to T * F$
(4) $T \to F$
(5) $F \to (E)$
(6) $F \to \mathbf{id}$

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

SLR(1) Grammar

- An LR parser using SLR(1) parsing tables for a grammar G is called as the SLR(1) parser for G.
- If a grammar G has an SLR(1) parsing table, it is called SLR(1) grammar (or SLR grammar in short).
- Every SLR grammar is unambiguous, but every unambiguous grammar is not a SLR grammar.

shift/reduce and reduce/reduce conflicts

- If a state does not know whether it will make a shift operation or reduction for a terminal, we say that there is a **shift/reduce conflict**.

- If a state does not know whether it will make a reduction operation using the production rule `i` or `j` for a terminal, we say that there is a **reduce/reduce conflict**.

- If the SLR parsing table of a grammar G has a conflict, we say that that grammar is not SLR grammar.

Conflict Example1

$S \rightarrow L{=}R$

$S \rightarrow R$

$L \rightarrow *R$

$L \rightarrow id$

$R \rightarrow L$

I_0: $S' \rightarrow .S$

$S \rightarrow .L{=}R$

$S \rightarrow .R$

$L \rightarrow .*R$

$L \rightarrow .id$

$R \rightarrow .L$

I_1: $S' \rightarrow S.$

I_2: $S \rightarrow L.{=}R$

$R \rightarrow L.$

I_3: $S \rightarrow R.$

I_4: $L \rightarrow *.R$

$R \rightarrow .L$

$L \rightarrow .*R$

$L \rightarrow .id$

I_5: $L \rightarrow id.$

I_6: $S \rightarrow L{=}.R$

$R \rightarrow .L$

$L \rightarrow .*R$

$L \rightarrow .id$

I_7: $L \rightarrow *R.$

I_8: $R \rightarrow L.$

I_9: $S \rightarrow L{=}R.$

Problem

FOLLOW(R)={=,$}

= → shift 6

→ reduce by $R \rightarrow L$

shift/reduce conflict

Conflict Example2

$S \rightarrow AaAb$

$S \rightarrow BbBa$

$A \rightarrow \varepsilon$

$B \rightarrow \varepsilon$

I_0: $S' \rightarrow .S$

$S \rightarrow .AaAb$

$S \rightarrow .BbBa$

$A \rightarrow .$

$B \rightarrow .$

Problem

FOLLOW(A)={a,b}

FOLLOW(B)={a,b}

a → reduce by $A \rightarrow \varepsilon$

a → reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

b → reduce by $A \rightarrow \varepsilon$

b → reduce by $B \rightarrow \varepsilon$

reduce/reduce conflict

Viable Prefixes

- If the stack holds α and the rest of the input is x, then a sequence of reductions will take αx to S. In terms of derivations, $S \overset{*}{\underset{rm}{\Rightarrow}} \alpha x$

- **The prefixes of right sentential forms that can appear on the stack of a shift-reduce parser are called viable prefixes.**
- They are defined as follows: a viable prefix is a prefix of a right-sentential form that does not continue past the right end of the rightmost handle of that sentential form.
- SLR parsing is based on the fact that LR(O) automata recognize viable prefixes. We say item $A \rightarrow \beta_1.\beta_2$ is valid for a viable prefix $\alpha\beta_1$ if there is a derivation

$$S' \overset{*}{\underset{rm}{\Rightarrow}} \alpha A w \underset{rm}{\Rightarrow} \alpha\beta_1\beta_2 w$$

Canonical LR(1) Items

- Recall that in the SLR method, state i calls for reduction by $A \rightarrow \alpha$ if the set of items I_i contains item $[A \rightarrow \alpha\cdot]$ and a is in FOLLOW(A) .
- **In some situations, however, when state i appears on top of the stack, the viable prefix βα on the stack is such that βA cannot be followed by "a" in any right-sentential form. Thus, the reduction by $A \rightarrow \alpha$ should be invalid on input "a" .**
- It is possible to carry more information in the state that will allow us to rule out some of these invalid reductions by $A \rightarrow \alpha$
- By redefining items to include a terminal symbol as a **second component** i.e. $[A \rightarrow \alpha.\beta, a]$ where $A \rightarrow \alpha\beta$ is a production and "a" is the terminal, known as LR(1) item.

Canonical LR(1) Items

- The lookahead has no effect in an item of the form [A→α.β, a] , where β is not ε,
- but an item of the form [A →α. , a] calls for a reduction by A →α only if the next input symbol is “a”.

Constructing LR(l) Sets of Items

```
SetOfItems CLOSURE(I) {
    repeat
        for ( each item [A → α·Bβ, a] in I )
            for ( each production B → γ in G′ )
                for ( each terminal b in FIRST(βa) )
                    add [B → ·γ, b] to set I;
    until no more items are added to I;
    return I;
}
```

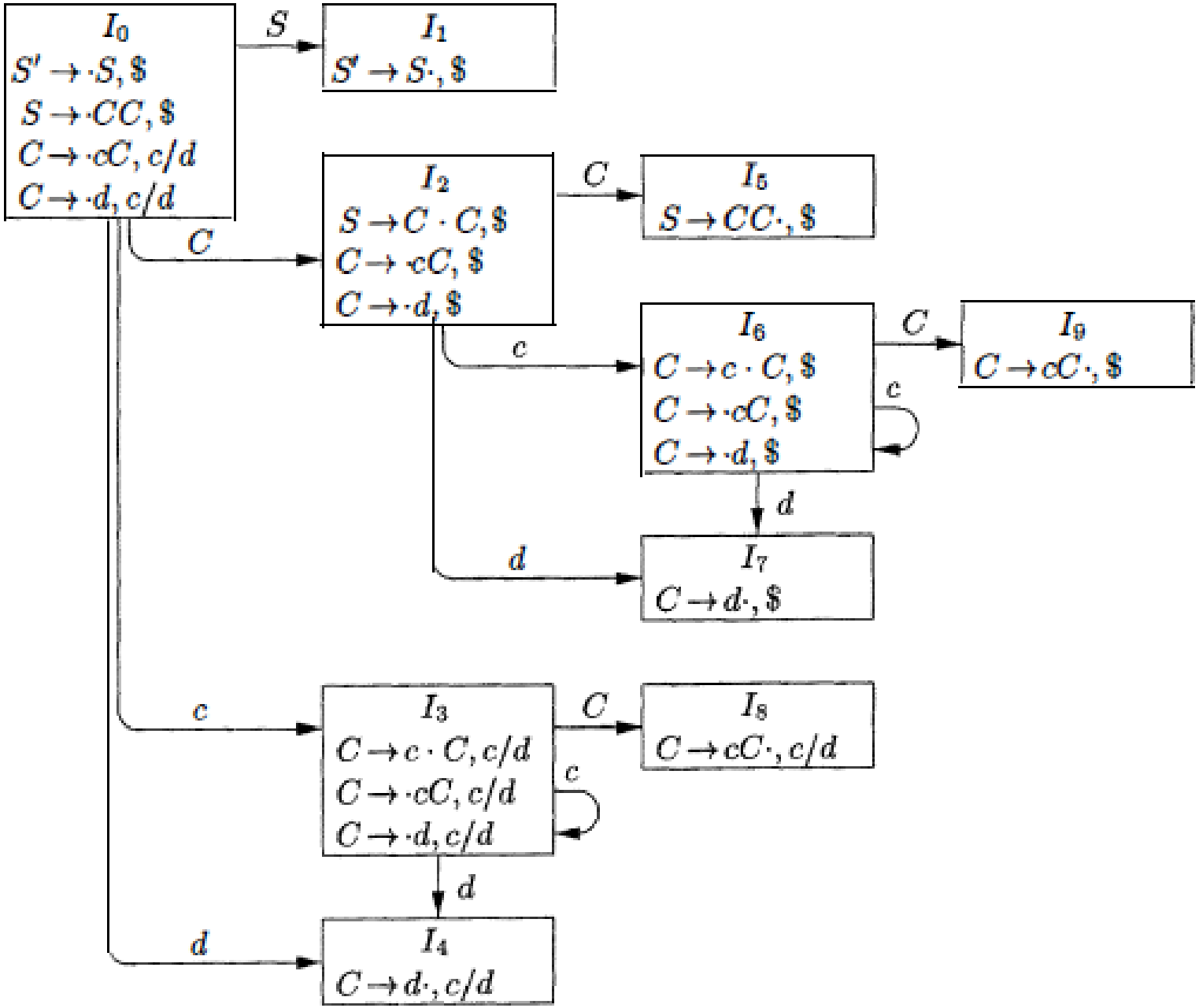
```
SetOfItems GOTO(I, X) {
    initialize J to be the empty set;
    for ( each item [A → α·Xβ, a] in I )
        add item [A → αX·β, a] to set J;
    return CLOSURE(J);
}
```

Constructing LR(I) Sets of Items

```
void items(G') {
    initialize C to CLOSURE({[S' → ·S, $]});
    repeat
        for ( each set of items I in C )
            for ( each grammar symbol X )
                if ( GOTO(I, X) is not empty and not in C )
                    add GOTO(I, X) to C;
    until no new sets of items are added to C;
}
```

$$
\begin{aligned}
S' &\rightarrow S \\
S &\rightarrow C\ C \\
C &\rightarrow c\ C \mid d
\end{aligned}
$$

STATE	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		



Canonical LR(I) Parsing Tables

1. Construct $C' = \{I_0, I_1, \cdots, I_n\}$, the collection of sets of LR(1) items for G'.
2. State i of the parser is constructed from I_i. The parsing action for state i is determined as follows.
   (a) If $[A \to \alpha \cdot a\beta, b]$ is in I_i and GOTO$(I_i, a) = I_j$, then set ACTION$[i, a]$ to "shift j." Here a must be a terminal.
   (b) If $[A \to \alpha \cdot, a]$ is in I_i, $A \neq S'$, then set ACTION$[i, a]$ to "reduce $A \to \alpha$."
   (c) If $[S' \to S \cdot, \$]$ is in I_i, then set ACTION$[i, \$]$ to "accept."

   If any conflicting actions result from the above rules, we say the grammar is not LR(1). The algorithm fails to produce a parser in this case.
3. The goto transitions for state i are constructed for all nonterminals A using the rule: If GOTO$(I_i, A) = I_j$, then GOTO$[i, A] = j$.
4. All entries not defined by rules (2) and (3) are made "error."
5. The initial state of the parser is the one constructed from the set of items containing $[S' \to \cdot S, \$]$.

Canonical LR(l) Parsing Tables

STATE	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

Canonical LR(l) Parsing Tables

- Every SLR(l) grammar is an LR(l) grammar
- But for an SLR(l) grammar the canonical LR parser may have more states than the SLR parser for the same grammar

Constructing LALR Parsing Tables

- LALR (lookahead LR)
- Tables obtained by it are considerably smaller than the canonical LR tables
- The SLR and LALR tables for a grammar always have the same number of states
- SLR and LALR tables typically several hundred states for a language like C, canonical LR table would typically have several thousand states

Constructing LALR Parsing Tables

1. Construct $C = \{I_0, I_1, \ldots, I_n\}$, the collection of sets of LR(1) items.

2. For each core present among the set of LR(1) items, find all sets having that core, and replace these sets by their union.

3. Let $C' = \{J_0, J_1, \ldots, J_m\}$ be the resulting sets of LR(1) items. The parsing actions for state i are constructed from J_i in the same manner as in Algorithm 4.56. If there is a parsing action conflict, the algorithm fails to produce a parser, and the grammar is said not to be LALR(1).

4. The GOTO table is constructed as follows. If J is the union of one or more sets of LR(1) items, that is, $J = I_1 \cap I_2 \cap \cdots \cap I_k$, then the cores of $\text{GOTO}(I_1, X)$, $\text{GOTO}(I_2, X), \ldots, \text{GOTO}(I_k, X)$ are the same, since $I_1, I_2, \ldots, I_k$ all have the same core. Let K be the union of all sets of items having the same core as $\text{GOTO}(I_1, X)$. Then $\text{GOTO}(J, X) = K$.

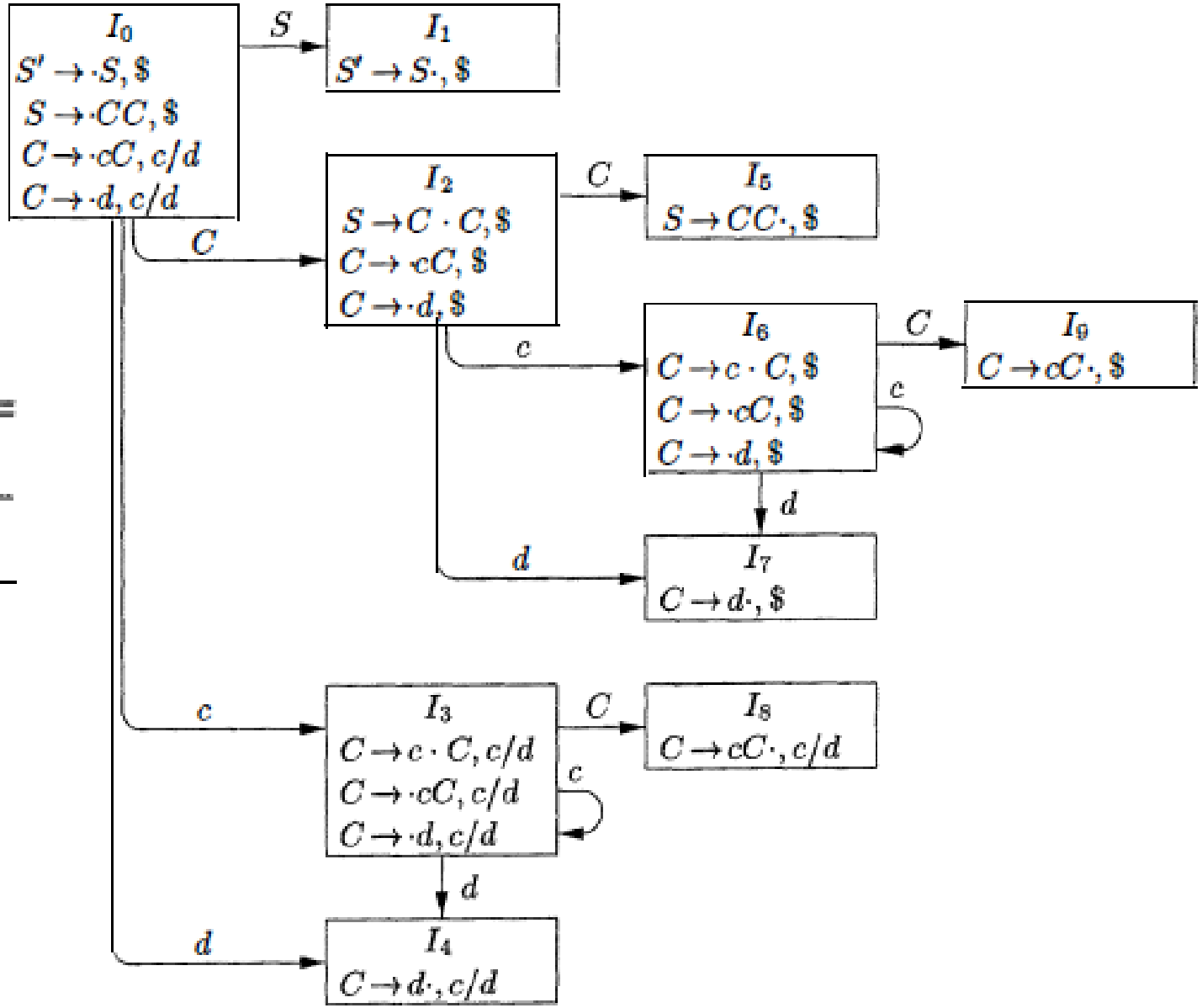
$$
\begin{array}{lcl}
S' & \rightarrow & S \\
S & \rightarrow & C\ C \\
C & \rightarrow & c\ C \mid d
\end{array}
$$

I_{36}: $C \rightarrow c \cdot C,\ c/d/\$$
$C \rightarrow \cdot cC,\ c/d/\$$
$C \rightarrow \cdot d,\ c/d/\$$

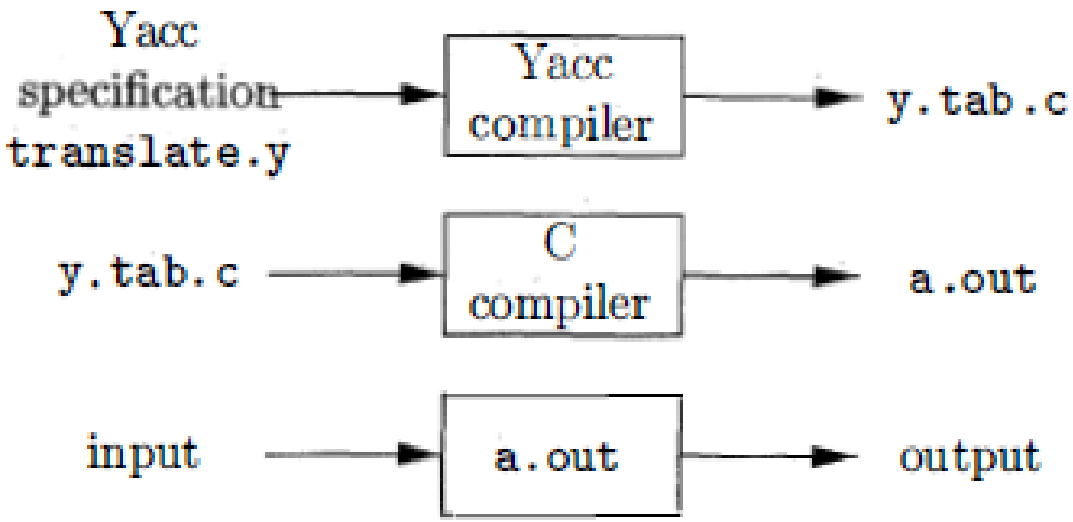
I_{47}: $C \rightarrow d \cdot,\ c/d/\$$

I_{89}: $C \rightarrow cC \cdot,\ c/d/\$$

STATE	ACTION			GOTO	
	c	d	\$	S	C
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		



The Parser Generator Yacc



declarations
%%
translation rules
%%
supporting C routines

The Parser Generator Yacc

$$
\begin{array}{lcl}
E & \rightarrow & E + T \mid T \\
T & \rightarrow & T * F \mid F \\
F & \rightarrow & (E) \mid \textbf{digit}
\end{array}
$$

```
%{
#include <ctype.h>
%}

%token DIGIT

%%
line   : expr '\n'          { printf("%d\n", $1); }
       ;
expr   : expr '+' term      { $$ = $1 + $3; }
       | term
       ;
term   : term '*' factor    { $$ = $1 * $3; }
       | factor
       ;
factor : '(' expr ')'       { $$ = $2; }
       | DIGIT
       ;
%%
yylex() {
    int c;
    c = getchar();
    if (isdigit(c)) {
        yylval = c-'0';
        return DIGIT;
    }
    return c;
}
```